

CISC Vs RISC



**Politecnico
di Torino**

R. Ferrero

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA)
License. To view a copy of this license, visit
<http://creativecommons.org/licenses/by-sa/3.0/>



Instruction set architecture (ISA)

- Many programming languages exist:
<https://www.tiobe.com/tiobe-index/>
 - High-level languages
 - Low-level languages
- A low-level language is closed to an instruction set architecture, which defines:
 - supported instructions
 - data types
 - registers
 - addressing modes

Register-Memory Architecture

- Operands of instructions can be immediates, or contained in memory or in registers
- If all operands can be in registers or in memory (or in combinations), the architecture is called **register-plus-memory**

Register-Register Architecture

- Also called **load-store** architecture
- Instructions are divided into:
 - **memory access** (load and store between memory and registers)
 - **ALU operations** (only between registers)
- Operands of ALU instructions are immediates or contained in registers

CISC

- CISC: Complex Instruction Set Computer
- The main purpose is to complete a task in as few assembly lines as possible
- Common CISC CPUs are:
 - x86 (1978) and x64 (2003) family, by Intel and AMD
 - Motorola 68000 series (1979)
 - IBM System/360 family (1964)
 - PDP-11 (1970) and VAX families (1977), by DEC

Key properties of CISC

- The length of the instructions is variable
 - Instructions of x86 family are between 1 and 6 bytes
- The processor architecture is more complex
- The number of clock cycles required for execution is variable
- Small code size

RISC

- RISC: Reduced Instruction Set Computer
- The main purpose is to make the processor as simple as possible
- Common RISC CPUs are:
 - ARM family (1985), designed by ARM Holdings
 - MIPS family (1984), by MIPS Technologies Inc.
 - RISC-V (2014)
 - SPARC family (1985), by Sun Microsystems
 - PowerPC (1991), by Apple, IBM, and Motorola

Key properties of RISC

- The instructions are simpler
- The length of the instructions is fixed
 - For RISC-V they have a fixed size of 32 bits
- The processor architecture is simpler
- The number of clock cycles required to execute an instruction is normally lower than in CISC
 - Optimistically 1 clock cycle
- Longer code length

A Practical Example

x86 code (CISC)

```
MOV AL, [SI]  
ADD AL, [BX]
```

RISC-V code (RISC)

```
LB t1, 0(t0)  
LB t3, 0(t2)  
ADD t4, t1, t3
```

A Practical Example

x86 code (CISC)

```
MOV AL, [SI]  
ADD AL, [BX]
```



RISC-V code (RISC)

Load into the AL register the value contained in the memory location pointed to by the SI register

```
ADD C4, C1, C3
```

A Practical Example

x86 code (CISC)

```
MOV AL, [SI]  
ADD AL, [BX]
```



RISC-V code (RISC)

Sum the value contained in the AL register with the value contained in the memory location pointed to by the BX register and save the result in AL

A Practical Example

x86 code (CISC)

Load into register t1 the value contained in the memory location equal to the sum of the value contained in register t0 and 0

RISC-V code (RISC)

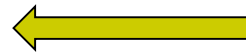
← LB t1, 0(t0)
LB t3, 0(t2)
ADD t4, t1, t3

A Practical Example

x86 code (CISC)

RISC-V code (RISC)

Load into register t3 the value contained in the memory location equal to the sum of the value contained in register t2 and 0



```
LB t1, 0(t0)
LB t3, 0(t2)
ADD t4, t1, t3
```

A Practical Example

x86 code (CISC)

```
MOV AL, [SI]
```

Sum the value contained in the t1 register with the value contained in the t3 register and save the result in t4

RISC-V code (RISC)

```
LB t1, 0(t0)
```

```
LB t3, 0(t2)
```

```
← ADD t4, t1, t3
```

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

```
LEA  BX,  VAR  
DEC  [BX]  
MOV  AX,  [BX]
```

x86 code (CISC)

```
LEA  BX,  VAR  
MOV  AX,  [BX]  
DEC  AX
```

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

```
LEA BX, VAR  
DEC [BX]  
MOV AX, [BX]
```



Load into register BX the memory address of variable VAR

```
MOV AX, [BX]  
DEC AX
```

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

x86 code (CISC)

LEA BX, VAR

DEC [BX] 

MOV AX, [BX]

Decrement by 1 the value
contained in the memory location
pointed to by the BX register

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

```
LEA BX, VAR
```

```
DEC [BX]
```

```
MOV AX, [BX]
```



x86 code (CISC)

```
LEA BX, VAR
```

Load into register AX the value contained in the memory location pointed to by the BX register

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

Load into register BX the
memory address of variable VAR

~~DEC [BX]~~

MOV AX, [BX]

x86 code (CISC)

← LEA BX, VAR

MOV AX, [BX]

DEC AX

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

x86 code (CISC)

LEA BX, VAR

Load into register AX the value contained in the memory location pointed to by the BX register



LEA BX, VAR
MOV AX, [BX]
DEC AX

Coding Optimization for Register-Memory Architectures

x86 code (CISC)

```
LEA BX, VAR  
DEC [BX]
```

Decrement by 1 the value
contained in the AX register

x86 code (CISC)

```
LEA BX, VAR  
MOV AX, [BX]  
DEC AX
```



Coding Optimization for Register-Memory Architectures

x86 code (CISC)

```
LEA BX, VAR  
DEC [BX]  
MOV AX, [BX]
```

x86 code (CISC)

```
LEA BX, VAR  
MOV AX, [BX]  
DEC AX
```



Coding Optimization for Register-Memory Architectures

x86 code (CISC)

```
LEA BX, VAR  
DEC [BX]  
MOV AX, [BX]
```



VS

x86 code (CISC)

```
LEA BX, VAR  
MOV AX, [BX]  
DEC AX
```



Coding Optimization for Register-Memory Architectures

- The order of instructions can affect the number of memory accesses
- The more they increase, the more it slows down execution performance
- Load-store architectures, by their nature, do not suffer from this problem

8086 addressing modes

- 1) Register mode: `MOV AX, BX`
- 2) Immediate mode: `MOV AX, 2000`
- 3) Displacement or direct mode: `MOV AX, [DISP]`
- 4) Register indirect mode: `MOV AX, [DI]`
- 5) Based indexed mode: `MOV AX, [BX + DI]`
- 6) Indexed mode: `MOV AX, [SI + 2000]`
- 7) Based mode: `MOV AX, [BP + 0100]`
- 8) Based indexed displacement mode:
`MOV AX, [SI + BP + 2000]`

RISC-V addressing modes

- ALU operations
 - Register addressing mode: `ADD t0, t1, t2`
 - Immediate addressing mode: `ADDI t0, t1, 10`
- Load / store operations
 - base + displacement addressing mode
`LW t2, 4(t1)`

ARM addressing modes

- ALU operations
 - Register addressing mode: `ADD r0, r1, r2`
 - Immediate addressing mode: `ADD r0, r1, #10`
- Load / store operations
 - Pre-indexed addressing
 - `LDR r2, [r0, r1, LSL #1]`
 - `LDR r2, [r0, #4]!`
 - Post-indexed addressing
 - `LDR r2, [r0], #4`

References

- Michael J. Flynn (1995). Computer architecture: pipelined and parallel processor design. pp. 9–12
- RISC vs CISC, <https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/>